



INFORME DEL TRABAJO

I. PORTADA

Tema:	TAREA 03 Listas
Unidad de Organización Curricular:	BÁSICA
Nivel y Paralelo:	Tercero - B
Alumno participante:	Camacho Monta Josue Jampier
	
Asignatura:	Estructura de Datos
Docente:	Ing. José Caiza, Mg.

INFORME DEL TRABAJO

2.1 Objetivos

General:

Implementar y analizar el funcionamiento de listas circulares en Java, aplicando sus propiedades de recorrido cíclico a distintos escenarios prácticos.

Específicos:

- Desarrollar las operaciones fundamentales sobre una lista circular simplemente enlazada.
- Aplicar la estructura circular a la simulación de algoritmos reales, como Round-Robin y el problema de Josephus.
- Evaluar las ventajas de una estructura circular frente a una lineal en un caso aplicado.

2.2 Instrucciones

1. Implemente en java una **lista simplemente enlazada** circular que permita realizar las siguientes operaciones:
 - Insertar al inicio
 - Insertar al final
 - Mostrar todos los elementos
 - Verificar si la lista está vacía
 - Contar el número de elementos

Requisito adicional:

Explique con un ejemplo por qué en una lista circular el último nodo debe apuntar al primero.

2. Inserción y eliminación controlada

Pregunta práctica:

Desarrolle un programa en java que implemente una lista circular y permita:

- Insertar un elemento en una posición específica
- Eliminar un elemento por posición
- Eliminar un elemento por valor
- Imprimir la lista antes y después de cada operación

Análisis solicitado:

Explique qué ocurre en los siguientes casos:



- lista vacía,
- lista con un solo nodo,
- lista con varios nodos.

3. Simulación de turnos Round-Robin

Pregunta práctica:

Construya un programa en java que simule la atención de procesos usando una **lista circular** bajo el algoritmo **Round-Robin**.

Considere lo siguiente:

- Cada proceso tiene: nombre y tiempo restante
- El quantum será de 2 unidades
- Si el proceso no termina, vuelve al final del ciclo
- Si termina, debe eliminarse de la lista

Solicitado:

Muestre paso a paso el orden de ejecución de los procesos y el estado de la lista después de cada turno.

4. Resolución del problema de Josephus

Pregunta práctica:

Implemente en java el problema de **Josephus** usando una lista circular.

Enunciado:

Dadas n personas ubicadas en círculo, elimine cada k -ésima persona hasta que quede solo una.

Solicitado:

- Probar con $n = 5$ y $k = 2$
- Probar con $n = 7$ y $k = 3$
- Mostrar el orden de eliminación
- Indicar quién es el superviviente final

Explique:

¿Por qué una lista circular es una estructura adecuada para este problema?

5. Caso aplicado: sistema de reproducción circular

Pregunta práctica:

Diseñe un programa en java que represente una **playlist musical circular**.

El sistema debe permitir:

- Agregar canciones al inicio y al final
- Mostrar la playlist completa
- Reproducir la siguiente canción
- Eliminar una canción por nombre
- Volver automáticamente a la primera canción al llegar al final

Análisis solicitado:

Explique qué ventaja tiene usar una lista circular en este caso frente a una lista lineal. La pregunta se fundamenta en las propiedades de recorrido cíclico y operaciones de inserción/eliminación de listas enlazadas.



2.3 Resultados obtenidos

Resultados obtenidos de la TAREA 03 Listas.

1. Ejercicio 1 — Lista simplemente enlazada circular

1.1. Ejercicio

Implemente en Java una lista simplemente enlazada circular que permita realizar las siguientes operaciones: insertar al inicio, insertar al final, mostrar todos los elementos, verificar si la lista está vacía y contar el número de elementos.

Requisito adicional: explicar con un ejemplo por qué en una lista circular el último nodo debe apuntar al primero.

1.2. Solución propuesta

Se implementa una estructura `Nodo1` (dato + referencia siguiente) y una clase `Ejercicio1_ListaCircular` que administra un puntero cabeza. A diferencia de un arreglo estático, aquí la estructura crece dinámicamente y el último nodo siempre apunta de regreso a la cabeza, cerrando el círculo.

1.3. ¿Qué es una lista circular?

Una lista circular es una estructura de datos donde:

- los datos que almacena son nodos enlazados de un mismo tipo;
- las operaciones que pueden realizarse son insertar, eliminar, recorrer y consultar;
- la regla que siempre debe cumplirse es que el último nodo nunca apunta a NULL, sino de vuelta al primer nodo (cabeza).

En este ejercicio la estructura es `Ejercicio1_ListaCircular`, cuyo único dato de estado es la referencia cabeza (privada a nivel de clase mediante campos static).

1.4. Relación entre requisitos y código

Requisito	Aplicación en el código
Insertar al inicio	método insertarInicio(): el nuevo nodo pasa a ser cabeza
Insertar al final	método insertarFinal(): se ubica el último nodo y se enlaza el nuevo
Mostrar todos los elementos	método mostrarLista(): recorrido do-while hasta volver a cabeza
Verificar si está vacía	método estaVacía(): retorna cabeza == null
Contar elementos	método contarElementos(): recorrido contando una vuelta completa
Menú de opciones	bucle do-while con switch en main()

1.5. Paso 1: Importar Scanner

Se importa `java.util.Scanner` para leer desde teclado las opciones del menú y los valores a insertar.

1.6. Paso 2: Crear la clase del nodo

Se crea la clase `Nodo1` con dos atributos:

- dato (tipo primitivo int)
- siguiente (referencia al propio tipo `Nodo1`).



Es el bloque básico de la estructura.

1.7. Paso 3: Construir la estructura circular

A diferencia del TDA de vehículos, aquí no existe herencia entre tipos de nodo. **Lo que reemplaza este paso es la construcción de la circularidad:** al insertar el primer elemento, el nodo se enlaza a sí mismo (nuevo.siguiete = nuevo), y en cada inserción posterior se ubica el último nodo (el que apunta a cabeza) para reconectarlo.

1.8. Paso 4: Lógica principal del algoritmo

El concepto aplicado aquí es el patrón de recorrido circular: se usa un ciclo do-while con la condición actual != cabeza (en vez de actual != null), porque en una lista circular nunca se llega a null.

1.9. Paso 5: Estado de la estructura

No se usa un arreglo estático; la capacidad es dinámica. **El estado del TDA se reduce a una sola referencia:** static Nodo1 cabeza, que inicia en null.

1.10. Paso 6: Registrar / construir elementos

El registro de elementos ocurre en insertarInicio() e insertarFinal(): ambos ubican el último nodo (el que apunta de vuelta a cabeza) y reconectan las referencias siguiente correspondientes.

1.11. Paso 7: Instanciar objetos

La instanciación ocurre con new Nodo1(valor) dentro de cada método de inserción; el objeto creado se integra de inmediato al círculo mediante las reasignaciones de siguiente.

1.12. Tipos de datos utilizados

Tipo	Categoría	Uso
int	Primitivo	Dato de cada nodo, contador de elementos
boolean	Primitivo	Resultado de estaVacía()
Nodo1	Referencia	Cada nodo de la lista y el puntero cabeza

1.13. Diagrama de clases

```
classDiagram
class Nodo1 {
+int dato
+Nodo1 siguiente
}
class Ejercicio1_ListaCircular {
-Nodo1 cabeza
+insertarInicio(int)
+insertarFinal(int)
+mostrarLista()
+estaVacía() boolean
+contarElementos() int
}
Ejercicio1_ListaCircular o-- Nodo1
Nodo1 --> Nodo1 : siguiente
```

1.14. Especificación de la estructura

Estado:

- **cabeza:** referencia al primer nodo (null si la lista está vacía)



UNIVERSIDAD TÉCNICA DE AMBATO
FACULTAD DE INGENIERÍA EN SISTEMAS ELECTRÓNICA E INDUSTRIAL
CARRERA DE SOFTWARE
CICLO ACADÉMICO: JULIO 2026 – DICIEMBRE 2026



Operaciones:

Método	Resultado
insertarInicio()	Agrega un nodo y lo convierte en la nueva cabeza
insertarFinal()	Agrega un nodo después del último, sin cambiar la cabeza
mostrarLista()	Imprime todos los datos en orden hasta regresar a la cabeza
estaVacía()	Indica si cabeza es null
contarElementos()	Devuelve cuántos nodos tiene la lista

Invariantes:

- El último nodo siempre tiene siguiente apuntando a cabeza (nunca a null).
- Si cabeza es null, la lista no tiene ningún nodo.
- Todo recorrido debe detenerse al volver a cabeza, nunca busca null.

1.15. Pseudocódigo

INICIO

cabeza <- null

REPETIR

mostrar menu

leer opcion

SEGUN opcion HACER

1: leer valor; insertarInicio(valor)

2: leer valor; insertarFinal(valor)

3: mostrarLista()

4: mostrar estaVacía()

5: mostrar contarElementos()

6: finalizar

FIN SEGUN

HASTA QUE opcion = 6

FIN

1.16. Prueba de escritorio

Paso	Operación	Estado de la lista	Cantidad
1	Crear la lista	cabeza = null	0
2	insertarFinal(10)	10 -> (vuelve a 10)	1
3	insertarFinal(20)	10 -> 20 -> (vuelve a 10)	2
4	insertarInicio(5)	5 -> 10 -> 20 -> (vuelve a 5)	3
5	contarElementos()	sin cambios	3

1.17. Ejemplo del requisito adicional

Ejemplo del requisito adicional: con la lista 5 -> 10 -> 20, si el último nodo (20) apuntara a null, mostrarLista() terminaría abruptamente y no habría forma de volver al inicio. Al apuntar de vuelta a 5, el recorrido puede repetirse indefinidamente, que es la propiedad que define a una lista circular.



2. Ejercicio 2 — Inserción y eliminación controlada

2.1. Ejercicio

Desarrolle un programa en Java que implemente una lista circular y permita: insertar un elemento en una posición específica, eliminar un elemento por posición, eliminar un elemento por valor e imprimir la lista antes y después de cada operación.

Análisis solicitado: explicar qué ocurre en los casos de lista vacía, lista con un solo nodo y lista con varios nodos.

2.2. Solución propuesta

Se reutiliza la estructura `Nodo2` (dato + siguiente) y se agrega lógica de recorrido con un puntero anterior para poder reconectar la lista al insertar o eliminar en cualquier posición, incluyendo los casos extremos de cabeza y de un único nodo.

2.3. ¿Qué es una lista circular?

La estructura administra:

- nodos con un dato entero;
- las operaciones `insertarEnPosicion`, `eliminarPorPosicion`, `eliminarPorValor` y `mostrarLista`;
- la regla de que la circularidad (último apunta a cabeza) debe mantenerse después de cada modificación.

2.4. Relación entre requisitos y código

Requisito	Aplicación en el código
Insertar en posición específica	insertarEnPosicion(valor, posicion): recorre hasta el nodo anterior a la posición
Eliminar por posición	eliminarPorPosicion(posicion): ubica el nodo anterior y lo reconecta al siguiente del eliminado
Eliminar por valor	eliminarPorValor(valor): recorre comparando cada dato hasta encontrar coincidencia
Imprimir antes y después	mostrarLista() se invoca explícitamente antes y después de cada operación en el menú
Insertar en posición específica	insertarEnPosicion(valor, posicion): recorre hasta el nodo anterior a la posición
Eliminar por posición	eliminarPorPosicion(posicion): ubica el nodo anterior y lo reconecta al siguiente del eliminado

2.5. Paso 1: Importar Scanner

Se importa `java.util.Scanner` para leer la opción del menú, el valor y la posición.

2.6. Paso 2: Crear la clase del nodo

La clase `Nodo2` mantiene la misma estructura básica:

- un dato primitivo (`int`) y una referencia siguiente.

2.7. Paso 3: Construir la estructura circular

En vez de herencia, el paso equivalente es el manejo de referencias dobles (`actual/anterior`) necesario para insertar o eliminar en medio del círculo sin perder la conexión hacia cabeza.



2.8. Paso 4: Lógica principal del algoritmo

En vez de polimorfismo, aquí el concepto clave es la validación de casos especiales:

- lista vacía, un solo nodo (el nodo apunta a sí mismo) y varios nodos, cada uno con lógica distinta dentro de los mismos métodos.

2.9. Paso 5: Estado de la estructura

No hay arreglo estático:

- el estado es dinámico y se reduce a static Nodo2 cabeza.

2.10. Paso 6: Registrar / construir elementos

insertarEnPosicion() ubica el nodo anterior a la posición pedida y reconecta nuevo.siguiente = actual.siguiente antes de actual.siguiente = nuevo, para no perder el resto de la cadena.

2.11. Paso 7: Instanciar objetos

La instanciación de cada elemento ocurre con new Nodo2(valor) dentro de insertarEnPosicion(); no se crean objetos hijos ni jerarquías adicionales.

2.12. Tipos de datos utilizados

Tipo	Categoría	Uso
int	Primitivo	Dato del nodo, posición, valor de búsqueda
boolean	Primitivo	Resultado de estaVacía()
Nodo2	Referencia	Cada nodo y los punteros cabeza/actual/anterior

2.13. Diagrama de clases

```
classDiagram
class Nodo2 {
+int dato
+Nodo2 siguiente
}
class Ejercicio2_InsercionEliminacion {
-Nodo2 cabeza
+insertarEnPosicion(int, int)
+eliminarPorPosicion(int)
+eliminarPorValor(int)
+mostrarLista()
}
Ejercicio2_InsercionEliminacion o-- Nodo2
```

2.14. Especificación de la estructura

Estado:

- **cabeza:** referencia al primer nodo (null si la lista está vacía)

Operaciones:

Método	Resultado
insertarEnPosicion()	Inserta un nodo en la posición indicada
eliminarPorPosicion()	Elimina el nodo ubicado en esa posición
eliminarPorValor()	Elimina el primer nodo cuyo dato coincida
mostrarLista()	Imprime el estado actual de la lista



Invariantes:

- **Lista vacía:** toda operación de eliminación debe rechazarse sin lanzar excepción (se valida con estaVacia()).
- **Un solo nodo:** cabeza.siguiete == cabeza; al eliminarlo la lista vuelve a cabeza = null.
- **Varios nodos:** se requiere localizar el nodo anterior al afectado para reconectar la cadena sin romper la circularidad.

2.15. Pseudocódigo

INICIO

cabeza <- null

REPETIR

mostrar menu

leer opcion

SEGUN opcion HACER

1: leer valor y posicion; mostrarLista(); insertarEnPosicion(); mostrarLista()

2: leer posicion; mostrarLista(); eliminarPorPosicion(); mostrarLista()

3: leer valor; mostrarLista(); eliminarPorValor(); mostrarLista()

4: mostrarLista()

5: finalizar

FIN SEGUN

HASTA QUE opcion = 5

FIN

2.16. Prueba de escritorio

Paso	Operación	Estado de la lista	Cantidad
1	insertarEnPosicion(10,0)	vacía	10 (vuelve a 10)
2	insertarEnPosicion(20,1)	10	10 -> 20
3	insertarEnPosicion(15,1)	10 -> 20	10 -> 15 -> 20
4	eliminarPorValor(15)	10 -> 15 -> 20	10 -> 20
5	eliminarPorPosicion(0)	10 -> 20	20 (vuelve a 20)

3. Ejercicio 3 — Simulación de turnos Round-Robin

3.1. Ejercicio

Construya un programa en Java que simule la atención de procesos usando una lista circular bajo el algoritmo Round-Robin. Cada proceso tiene nombre y tiempo restante. El quantum será de 2 unidades. Si el proceso no termina, vuelve al final del ciclo; si termina, debe eliminarse de la lista.

Solicitado: mostrar paso a paso el orden de ejecución de los procesos y el estado de la lista después de cada turno.

3.2. Solución propuesta

Se crea una clase ProcesoNodo (nombre + tiempoRestante + siguiente) administrada como cola circular. En cada turno se descuenta el quantum al proceso actual; si su tiempo llega a 0 se elimina del círculo, y si no, se reubica al final de forma natural avanzando el puntero.

3.3. ¿Qué es una lista circular?

La estructura administra:

- procesos con nombre y tiempo restante;
- las operaciones agregarProceso, simular y mostrarEstado;



- **la regla de que un proceso nunca se pierde:** se ejecuta en orden circular hasta que su tiempo restante llega a cero.

3.4. Relación entre requisitos y código

Requisito	Aplicación en el código
Proceso con nombre y tiempo restante	Clase ProcesoNodo con atributos String nombre e int tiempoRestante
Quantum de 2 unidades	Constante static final int QUANTUM = 2
Vuelve al final si no termina	Se avanzan los punteros actual/anterior sin eliminar el nodo (circularidad natural)
Se elimina si termina	Se reconecta anterior.siguiente saltando el nodo actual
Mostrar paso a paso	Método mostrarEstado() se llama después de cada turno dentro de simular()
Proceso con nombre y tiempo restante	Clase ProcesoNodo con atributos String nombre e int tiempoRestante

3.5. Paso 1: Importar Scanner

Se importa `java.util.Scanner` para leer el nombre y el tiempo de cada proceso desde el menú.

3.6. Paso 2: Crear la clase del nodo

La clase **ProcesoNodo** combina dos tipos de datos primitivos distintos (String no es primitivo, pero int sí) junto con la referencia siguiente.

3.7. Paso 3: Construir la estructura circular

En vez de herencia entre tipos de nodo, aquí el concepto aplicado es la cola circular con quantum: cada vuelta al círculo representa un turno del sistema operativo simulado.

3.8. Paso 4: Lógica principal del algoritmo

En vez de polimorfismo, el comportamiento variable está en la decisión if (tiempoRestante <= 0): un mismo bucle produce dos caminos distintos (eliminar o reubicar) según el estado del proceso actual.

3.9. Paso 5: Estado de la estructura

No existe arreglo estático de tamaño fijo; la cantidad de procesos es dinámica y se representa únicamente con la referencia cabeza y el puntero anterior usado para eliminar.

3.10. Paso 6: Registrar / construir elementos

agregarProceso() funciona igual que insertarFinal() de los ejercicios anteriores: ubica el último nodo del círculo y engancha el nuevo proceso antes de cerrar de nuevo el círculo hacia cabeza.

3.11. Paso 7: Instanciar objetos

Cada proceso se instancia con `new ProcesoNodo(nombre, tiempo)` al elegir la opción 'Agregar proceso' del menú.

3.12. Tipos de datos utilizados

Tipo	Categoría	Uso
String	Referencia	nombre del proceso
int	Primitivo	tiempoRestante, QUANTUM, contador de turnos



ProcesoNodo	Referencia	cada proceso y los punteros cabeza/actual/anterior
-------------	------------	---

3.13. Diagrama de clases

```

classDiagram
class ProcesoNodo {
+String nombre
+int tiempoRestante
+ProcesoNodo siguiente
}
class Ejercicio3_RoundRobin {
-ProcesoNodo cabeza
-int QUANTUM
+agregarProceso(String, int)
+simular()
+mostrarEstado()
}
Ejercicio3_RoundRobin o--ProcesoNodo
  
```

3.14. Especificación de la estructura

Estado:

- **cabeza:** referencia al proceso que sigue en turno
- **QUANTUM = 2:** tiempo máximo asignado por turno

Operaciones:

Método	Resultado
agregarProceso()	Añade un proceso al final de la cola circular
simular()	Ejecuta turnos hasta que todos los procesos terminen
mostrarEstado()	Imprime la cola de procesos restante y su tiempo
agregarProceso()	Añade un proceso al final de la cola circular

Invariantes:

- tiempoRestante nunca es negativo (se descuenta con $\text{Math.min}(\text{QUANTUM}, \text{tiempoRestante})$).
- Un proceso se elimina exactamente cuando su tiempoRestante llega a 0.
- **El orden de atención respeta FIFO circular:** el siguiente en ejecutar siempre es el que sigue al actual.

3.15. Pseudocódigo

INICIO

cabeza <- null

agregar procesos (nombre, tiempo)

actual <- cabeza

MIENTRAS existan procesos HACER

ejecutar actual por $\min(\text{QUANTUM}, \text{tiempoRestante})$

SI tiempoRestante = 0 ENTONCES

eliminar actual de la lista

SINO

avanzar actual al siguiente (vuelve al final del ciclo)

FIN SI

mostrar estado de la lista



FIN MIENTRAS
FIN

3.16. Prueba de escritorio

Paso	Operación	Estado de la lista	Cantidad
1	P1 (tiempo 5)	2 (queda 3)	P2:3 -> P3:4 -> P1:3
2	P2 (tiempo 3)	2 (queda 1)	P3:4 -> P1:3 -> P2:1
3	P3 (tiempo 4)	2 (queda 2)	P1:3 -> P2:1 -> P3:2
4	P1 (tiempo 3)	2 (queda 1)	P2:1 -> P3:2 -> P1:1
5	P2 (tiempo 1)	1 (queda 0)	P3:2 -> P1:1 (P2 termina y sale)

4. Ejercicio 4 — Resolución del problema de Josephus

4.1. Ejercicio

Implemente en Java el problema de Josephus usando una lista circular. Dadas n personas ubicadas en círculo, elimine cada k -ésima persona hasta que quede solo una.

Solicitado: probar con $n = 5$, $k = 2$ y $n = 7$, $k = 3$; mostrar el orden de eliminación e indicar quién es el superviviente final. Explicar por qué una lista circular es adecuada para este problema.

4.2. Solución propuesta

Se arma un círculo de n nodos (PersonaNodo) numerados de 1 a n . Con dos punteros, actual y anterior, se avanza $k-1$ posiciones para llegar a la k -ésima persona, se elimina reconectando anterior.siguiente, y se repite hasta que solo quede un nodo apuntando a sí mismo.

4.3. ¿Qué es una lista circular?

La estructura administra:

- (n personas identificadas por un id entero;
- la operación `resolverJosephus(n, k)`;
- la regla de que el conteo debe recorrer el círculo de forma indefinida (sin fin natural) hasta que quede un único sobreviviente.

4.4. Relación entre requisitos y código

Requisito	Aplicación en el código
n personas en círculo	bucle for que crea n nodos y cierra el círculo con <code>actual.siguiente = cabeza</code>
Eliminar cada k-ésima persona	bucle for($i=1$; $i < k$; $i++$) que avanza actual y anterior antes de cada eliminación
Repetir hasta que quede una sola	while (<code>actual.siguiente != actual</code>)
Mostrar orden de eliminación	<code>System.out.print(actual.id)</code> antes de reconectar la lista
Indicar el superviviente	se imprime <code>actual.id</code> al salir del bucle



4.5. Paso 1: Importar Scanner

Se importa `java.util.Scanner` para leer `n` y `k` cuando el usuario elige la opción personalizada del menú.

4.6. Paso 2: Crear la clase del nodo

La clase `PersonaNodo` es la más simple de los cinco ejercicios: solo contiene un `id` entero y la referencia siguiente, ya que el problema no requiere más información por persona.

4.7. Paso 3: Construir la estructura circular

En vez de herencia, el paso equivalente es la construcción explícita del círculo completo antes de iniciar el algoritmo: un bucle enlaza secuencialmente `n` nodos y el último se conecta de regreso al primero.

4.8. Paso 4: Lógica principal del algoritmo

En vez de polimorfismo, el comportamiento distinto surge de los valores de `n` y `k`: la misma función `resolverJosephus()` produce resultados distintos según los parámetros de entrada, sin necesidad de sobrescribir ningún método.

4.9. Paso 5: Estado de la estructura

No hay arreglo estático ni capacidad fija: el círculo se arma dinámicamente según el `n` ingresado por el usuario.

4.10. Paso 6: Registrar / construir elementos

No existe un método 'registrar' como tal; el registro ocurre en el propio bucle de construcción del círculo, antes de ejecutar el algoritmo de eliminación.

4.11. Paso 7: Instanciar objetos

Cada persona se instancia con `new PersonaNodo(i)` dentro del `for` que arma el círculo, para `i` entre 1 y `n`.

4.12. Tipos de datos utilizados

Tipo	Categoría	Uso
int	Primitivo	id de cada persona, valores de <code>n</code> y <code>k</code>
PersonaNodo	Referencia	cada persona del círculo y los punteros actual/anterior

4.13. Diagrama de clases

```
classDiagram
class PersonaNodo {
+int id
+PersonaNodo siguiente
}
class Ejercicio4_Josephus {
+resolverJosephus(int, int)
}
Ejercicio4_Josephus o--PersonaNodo
```



4.14. Especificación de la estructura

Estado:

- círculo de n nodos numerados $1..n$, cerrado sobre sí mismo

Operaciones:

Método	Resultado
resolverJosephus(n, k)	Construye el círculo, elimina cada k -ésima persona e imprime el orden y el superviviente

Invariantes:

- Mientras haya más de un nodo, `actual.siguiente` nunca es null.
- El bucle termina exactamente cuando `actual.siguiente == actual` (un solo nodo restante).
- El total de personas eliminadas siempre es $n - 1$.

4.15. Pseudocódigo

INICIO

leer n, k

construir círculo de n personas ($1..n$)

`anterior` <- último nodo, `actual` <- cabeza

MIENTRAS `actual.siguiente != actual` HACER

REPETIR $k-1$ VECES: `anterior` <- `actual`; `actual` <- `actual.siguiente`

mostrar id de `actual` (eliminado)

`anterior.siguiente` <- `actual.siguiente`

`actual` <- `anterior.siguiente`

FIN MIENTRAS

mostrar id de `actual` como superviviente

FIN

4.16. Prueba de escritorio

Paso	Operación	Estado de la lista	Cantidad
$n = 5, k = 2$	2, 4, 1, 5	3	$n = 5, k = 2$
$n = 7, k = 3$	3, 6, 2, 7, 5, 1	4	$n = 7, k = 3$

4.17. Justificación solicitada

Una lista circular es adecuada para Josephus porque el algoritmo necesita recorrer el grupo dando vueltas repetidas sin un final predefinido: no se sabe de antemano cuántas vueltas completas se darán antes de que queden pocas personas. Una lista lineal obligaría a reiniciar manualmente el recorrido cada vez que se llega al final; la lista circular resuelve esto de forma natural, ya que el nodo siguiente después del último siempre existe y apunta de regreso al inicio del grupo restante.



5. Ejercicio 5 — Caso aplicado: sistema de reproducción circular

5.1. Ejercicio

Diseñe un programa en Java que represente una playlist musical circular. El sistema debe permitir agregar canciones al inicio y al final, mostrar la playlist completa, reproducir la siguiente canción, eliminar una canción por nombre y volver automáticamente a la primera canción al llegar al final.

Análisis solicitado: explicar qué ventaja tiene usar una lista circular frente a una lista lineal.

5.2. Solución propuesta

Se crea la clase `CancionNodo` (nombre + siguiente) y se agrega un segundo puntero, `actual`, que representa la canción que se está reproduciendo en un momento dado, independiente de cabeza (que solo marca el inicio de la playlist).

5.3. ¿Qué es una lista circular?

La estructura administra:

- canciones identificadas por su nombre;
- las operaciones `agregarInicio`, `agregarFinal`, `mostrarPlaylist`, `reproducirSiguiente` y `eliminarPorNombre`;
- **la regla de que el recorrido nunca debe detenerse:** al llegar a la última canción, la siguiente reproducción vuelve automáticamente a la primera.

5.4. Relación entre requisitos y código

Requisito	Aplicación en el código
Agregar al inicio y al final	<code>agregarInicio()</code> y <code>agregarFinal()</code> , igual patrón que en los ejercicios anteriores
Mostrar la playlist completa	<code>mostrarPlaylist()</code> : recorrido <code>do-while</code> marcando la canción en reproducción
Reproducir la siguiente canción	<code>reproducirSiguiente()</code> : <code>actual = actual.siguiente</code>
Eliminar por nombre	<code>eliminarPorNombre()</code> : busca por <code>String.equals()</code> y reconecta la lista
Volver automáticamente al inicio	propiedad inherente de la circularidad: el siguiente de la última canción ya es cabeza

5.5. Paso 1: Importar Scanner

Se importa `java.util.Scanner` para leer el nombre de las canciones desde el menú.

5.6. Paso 2: Crear la clase del nodo

La clase `CancionNodo` guarda un nombre (`String`) y la referencia siguiente; es estructuralmente igual a los nodos anteriores, cambiando solo el tipo de dato almacenado.

5.7. Paso 3: Construir la estructura circular

En vez de herencia, aquí se introduce un segundo puntero de estado (`actual`) además de cabeza, necesario porque la 'canción que suena' y la 'primera canción de la lista' son conceptos distintos que deben coexistir.

5.8. Paso 4: Lógica principal del algoritmo

En vez de polimorfismo, el comportamiento distinto entre reproducir y mostrar surge de qué puntero se usa en cada recorrido: `mostrarPlaylist()` recorre desde cabeza, mientras que `reproducirSiguiente()` solo avanza `actual` una posición.



5.9. Paso 5: Estado de la estructura

No hay arreglo estático ni capacidad fija: la playlist crece o se reduce dinámicamente según se agreguen o eliminen canciones.

5.10. Paso 6: Registrar / construir elementos

agregarInicio() y agregarFinal() siguen el mismo patrón de registro que en el ejercicio

1: ubicar el último nodo del círculo y reconectar las referencias siguiente para incluir la nueva canción.

5.11. Paso 7: Instanciar objetos

Cada canción se instancia con `new CancionNodo(nombre)` al elegir agregar al inicio o al final desde el menú.

5.12. Tipos de datos utilizados

Tipo	Categoría	Uso
String	Referencia	nombre de cada canción
CancionNodo	Referencia	cada canción y los punteros cabeza/actual

5.13. Diagrama de clases

```
classDiagram
class CancionNodo {
+String nombre
+CancionNodo siguiente
}
class Ejercicio5_PlaylistCircular {
-CancionNodo cabeza
-CancionNodo actual
+agregarInicio(String)
+agregarFinal(String)
+mostrarPlaylist()
+reproducirSiguiente()
+eliminarPorNombre(String)
}
Ejercicio5_PlaylistCircular o--CancionNodo
```

5.14. Especificación de la estructura

Estado:

- **cabeza:** referencia a la primera canción de la playlist
- **actual:** referencia a la canción que se está reproduciendo

Operaciones:

Método	Resultado
agregarInicio()	Agrega una canción y la vuelve la nueva cabeza
agregarFinal()	Agrega una canción al final de la playlist
mostrarPlaylist()	Imprime todas las canciones marcando la actual
reproducirSiguiente()	Avanza actual a la siguiente canción (vuelve a cabeza si estaba en la última)
eliminarPorNombre()	Elimina la canción indicada de la playlist



Invariantes:

- actual siempre apunta a una canción válida dentro del círculo, o es null si la playlist está vacía.
- Si se elimina la canción que estaba sonando, actual pasa automáticamente a la siguiente.
- El recorrido de reproducción nunca termina: no existe un 'final' explícito en la playlist.

5.15. Pseudocódigo

INICIO

cabeza <- null, actual <- null

REPETIR

mostrar menu

leer opcion

SEGUN opcion HACER

1: leer nombre; agregarInicio(nombre)

2: leer nombre; agregarFinal(nombre)

3: mostrarPlaylist()

4: reproducirSiguiente()

5: leer nombre; eliminarPorNombre(nombre)

6: finalizar

FIN SEGUN

HASTA QUE opcion = 6

FIN

5.16. Prueba de escritorio

Paso	Operación	Estado de la lista	Cantidad
1	agregarFinal("A")	Playlist: A (reproduciendo)	1
2	agregarFinal("B")	Playlist: A (reproduciendo) -> B	2
3	agregarFinal("C")	Playlist: A (reproduciendo) -> B -> C	3
4	reproducirSiguiente() x2	Playlist: A -> B -> C (reproduciendo)	4
5	reproducirSiguiente()	Vuelve automáticamente a A (reproduciendo)	5

5.17. Justificación solicitada

Frente a una lista lineal, la ventaja de la lista circular es que el recorrido de reproducción nunca se interrumpe: en una lista lineal, al llegar a la última canción, 'siguiente' sería null y el programa tendría que detectar ese caso y reiniciar manualmente el recorrido. En la lista circular esa lógica adicional no es necesaria, porque el último nodo ya apunta de regreso al primero; el ciclo de reproducción es una propiedad natural de la estructura, no algo que haya que programar aparte.

2.4 Conclusiones

- Las operaciones básicas de una lista circular exigen comparar el puntero con la cabeza en lugar de con null, requiriendo mayor cuidado al programar.
- La circularidad simplifica la lógica de "volver al inicio" en algoritmos de recorrido cíclico continuo, como Round-Robin y Josephus.
- Frente a una lista lineal, elimina la necesidad de reiniciar manualmente el recorrido, haciendo el código más simple en casos como la reproducción continua.



2.5 Recomendaciones

- Prestar atención especial a los casos límite, ya que son los que más errores generan al implementar listas circulares.
- Utilizar diagramas o pruebas de escritorio antes de codificar, para visualizar cómo se reconectan los nodos en cada operación.
- Reforzar la práctica con más problemas clásicos para afianzar el manejo de punteros en estructuras cíclicas.

2.6 Referencias bibliográficas

[1] Baeldung, "Reduce Object Header Size and Save Memory in Java 25," Baeldung, may 2026. Accedido el 30 de agosto de 2026. [En línea]. Disponible: <https://www.baeldung.com/java-object-header-reduced-size-save-memory>

[2] "The Cost of Garbage Collection for State Machine Replication," arXiv preprint, arXiv:2405.11182, 2024. Accedido el 30 de agosto de 2026. [En línea]. Disponible: <https://arxiv.org/pdf/2405.11182>.

2.7 Anexos

EVIDENCIA DE LOS CÓDIGOS Y SUS RESPECTIVAS EJECUCIONES

```
1 import java.util.Scanner;
2
3 class Nodol {
4     int dato;
5     Nodol siguiente;
6
7     public Nodol(int dato) {
8         this.dato = dato;
9         this.siguiente = null;
10    }
11 }
12
13 public class Ejercicio1_ListaCircular {
14
15     static Nodol cabeza;
16
17     // Retorna true si la lista no tiene nodos
18     public static boolean estaVacia() {
19         return cabeza == null;
20     }
21
22     // Inserta un nuevo nodo y lo convierte en la nueva cabeza
23     public static void insertarInicio(int valor) {
24         Nodol nuevo = new Nodol(valor);
25         if (estaVacia()) {
26             cabeza = nuevo;
27             nuevo.siguiente = cabeza;
28             return;
29         }
30         Nodol actual = cabeza;
31         while (actual.siguiente != cabeza) {
32             actual = actual.siguiente;
33         }
34         nuevo.siguiente = cabeza;
35         actual.siguiente = nuevo;
36         cabeza = nuevo;
37     }
38
39     // Inserta un nuevo nodo al final, sin cambiar la cabeza
40     public static void insertarFinal(int valor) {
41         Nodol nuevo = new Nodol(valor);
42         if (estaVacia()) {
43             cabeza = nuevo;
44             nuevo.siguiente = cabeza;
45             return;
46         }
47         Nodol actual = cabeza;
48         while (actual.siguiente != cabeza) {
49             actual = actual.siguiente;
50         }
51         nuevo.siguiente = cabeza;
52         actual.siguiente = nuevo;
53     }
54 }
```

Figura 1. Código Java del Ejercicio 1.



UNIVERSIDAD TÉCNICA DE AMBATO
FACULTAD DE INGENIERÍA EN SISTEMAS ELECTRÓNICA E INDUSTRIAL
CARRERA DE SOFTWARE
CICLO ACADÉMICO: JULIO 2026 – DICIEMBRE 2026



```
51     actual.siguiente = nuevo;
52     nuevo.siguiente = cabeza;
53 }
54
55 // Recorre la lista hasta volver a la cabeza (no hay NULL, es circular)
56 public static void mostrarLista() {
57     if (estaVacia()) {
58         System.out.println(x: "Lista vacia");
59         return;
60     }
61     Nodo1 actual = cabeza;
62     do {
63         System.out.print(actual.dato + " -> ");
64         actual = actual.siguiente;
65     } while (actual != cabeza);
66     System.out.println("vuelve a " + cabeza.dato + "");
67 }
68
69 // Cuenta los nodos recorriendo la lista una sola vuelta
70 public static int contarElementos() {
71     if (estaVacia()) return 0;
72     int contador = 0;
73     Nodo1 actual = cabeza;
74     do {
75         contador++;
76         actual = actual.siguiente;
77     } while (actual != cabeza);
78     return contador;
79 }
80
81 Run | Debug
82 public static void main(String[] args) {
83     Scanner sc = new Scanner(System.in);
84     int opcion, valor;
85
86     do {
87         System.out.println(x: "\n-----");
88         System.out.println(x: "LISTA CIRCULAR");
89         System.out.println(x: "-----");
90         System.out.println(x: "1. Insertar al inicio");
91         System.out.println(x: "2. Insertar al final");
92         System.out.println(x: "3. Mostrar lista");
93         System.out.println(x: "4. Verificar si esta vacia");
94         System.out.println(x: "5. Contar elementos");
95         System.out.println(x: "6. Salir");
96         System.out.print(s: "Seleccione una opcion: ");
97         opcion = sc.nextInt();
98
99         switch (opcion) {
100             case 1:
101                 System.out.print(s: "Ingrese valor: ");
102                 valor = sc.nextInt();
103                 insertarInicio(valor);
104                 break;
105             case 2:
106                 System.out.print(s: "Ingrese valor: ");
107                 valor = sc.nextInt();
108                 insertarFinal(valor);
109                 break;
110             case 3:
111                 mostrarLista();
112                 break;
113             case 4:
114                 System.out.println("Esta vacia? " + estaVacia());
115                 break;
116             case 5:
117                 System.out.println("Numero de elementos: " + contarElementos());
118                 break;
119             case 6:
120                 System.out.println(x: "Saliendo...");
121                 break;
122             default:
123                 System.out.println(x: "Opcion invalida");
124         }
125     } while (opcion != 6);
126
127     sc.close();
128 }
```

Figura 2. Código Java del Ejercicio 1.



UNIVERSIDAD TÉCNICA DE AMBATO
FACULTAD DE INGENIERÍA EN SISTEMAS ELECTRÓNICA E INDUSTRIAL
CARRERA DE SOFTWARE
CICLO ACADÉMICO: JULIO 2026 – DICIEMBRE 2026



```
-----  
LISTA CIRCULAR  
-----  
1. Insertar al inicio  
2. Insertar al final  
3. Mostrar lista  
4. Verificar si esta vacia  
5. Contar elementos  
6. Salir  
Seleccione una opcion: 1  
Ingrese valor: 120  
  
-----  
LISTA CIRCULAR  
-----  
1. Insertar al inicio  
2. Insertar al final  
3. Mostrar lista  
4. Verificar si esta vacia  
5. Contar elementos  
6. Salir  
Seleccione una opcion: 2  
Ingrese valor: 36  
  
-----  
LISTA CIRCULAR  
-----  
1. Insertar al inicio  
2. Insertar al final  
3. Mostrar lista  
4. Verificar si esta vacia  
5. Contar elementos  
6. Salir  
Seleccione una opcion: 3  
120 -> 36 -> (vuelve a 120)  
  
-----  
LISTA CIRCULAR  
-----  
1. Insertar al inicio  
2. Insertar al final  
3. Mostrar lista  
4. Verificar si esta vacia  
5. Contar elementos  
6. Salir  
Seleccione una opcion: 4  
Esta vacia? false  
  
-----  
LISTA CIRCULAR  
-----  
1. Insertar al inicio  
2. Insertar al final  
3. Mostrar lista  
4. Verificar si esta vacia  
5. Contar elementos  
6. Salir  
Seleccione una opcion: 5  
Numero de elementos: 2  
  
-----  
LISTA CIRCULAR  
-----  
1. Insertar al inicio  
2. Insertar al final  
3. Mostrar lista  
4. Verificar si esta vacia  
5. Contar elementos  
6. Salir  
Seleccione una opcion: 6  
Saliendo...  
PS C:\Users\Jampier_M\Documents\Tarea_Listas\Tarea_Listas\scr_Java> |
```

Figura 3. *Ejecución del Ejercicio 1.*



UNIVERSIDAD TÉCNICA DE AMBATO
FACULTAD DE INGENIERÍA EN SISTEMAS ELECTRÓNICA E INDUSTRIAL
CARRERA DE SOFTWARE
CICLO ACADÉMICO: JULIO 2026 – DICIEMBRE 2026



```
1 import java.util.Scanner;
2
3 class Node {
4     int dato;
5     Node siguiente;
6
7     public Node(int dato) {
8         this.dato = dato;
9         this.siguiente = null;
10    }
11 }
12
13 public class Ejercicio_InsercionEliminacion {
14
15     static Node cabeza;
16
17     // Retorna true si la lista no tiene nodos
18     public static boolean estaVacia() {
19         return cabeza == null;
20     }
21
22     // Recorre e imprime la lista hasta volver a la cabeza
23     public static void mostrarLista() {
24         if (estaVacia()) {
25             System.out.println("Lista vacía");
26             return;
27         }
28         Node actual = cabeza;
29         do {
30             System.out.print(actual.dato + " -> ");
31             actual = actual.siguiente;
32         } while (actual != cabeza);
33         System.out.println("(vuelve a " + cabeza.dato + ")");
34     }
35
36     // Insertar en una posición específica (0 = inicio)
37     public static void insertarPorPosicion(int valor, int posicion) {
38         Node nuevo = new Node(valor);
39
40         if (estaVacia() || posicion == 0) {
41             if (estaVacia()) {
42                 cabeza = nuevo;
43                 nuevo.siguiente = cabeza;
44             } else {
45                 Node ultimo = cabeza;
46                 while (ultimo.siguiente != cabeza) {
47                     ultimo = ultimo.siguiente;
48                 }
49                 nuevo.siguiente = cabeza;
50                 ultimo.siguiente = nuevo;
51                 cabeza = nuevo;
52             }
53             return;
54         }
55
56         Node actual = cabeza;
57         int i = 0;
58         // se busca el nodo anterior a la posición pedida
59         while (i < posicion - 1 && actual.siguiente != cabeza) {
60             actual = actual.siguiente;
61             i++;
62         }
63         nuevo.siguiente = actual.siguiente;
64         actual.siguiente = nuevo;
65     }
66
67     // Elimina el nodo en la posición dada (0 = primer nodo); reconecta al vecino
68     public static void eliminarPorPosicion(int posicion) {
69         if (estaVacia()) {
70             System.out.println("No se puede eliminar, la lista está vacía");
71             return;
72         }
73
74         // caso: un solo nodo
75         if (cabeza.siguiente == cabeza) {
76             cabeza = null;
77             return;
78         }
79
80         if (posicion == 0) {
81             Node ultimo = cabeza;
82             while (ultimo.siguiente != cabeza) {
83                 ultimo = ultimo.siguiente;
84             }
85             cabeza = cabeza.siguiente;
86             ultimo.siguiente = cabeza;
87             return;
88         }
89
90         Node anterior = cabeza;
91         int i = 0;
92         while (i < posicion - 1 && anterior.siguiente != cabeza) {
93             anterior = anterior.siguiente;
94             i++;
95         }
96         Node aeliminar = anterior.siguiente;
97         anterior.siguiente = aeliminar.siguiente;
98     }
99
100     // Busca el nodo con ese valor y lo desconecta de la lista
101     public static void eliminarPorValor(int valor) {
102         if (estaVacia()) {
103             System.out.println("No se puede eliminar, la lista está vacía");
104             return;
105         }
106
107         // caso: un solo nodo
108         if (cabeza.siguiente == cabeza) {
109             if (cabeza.dato == valor) {
110                 cabeza = null;
111             } else {
112                 System.out.println("Valor no encontrado");
113             }
114             return;
115         }
116
117         Node actual = cabeza;
118         Node anterior = null;
119
120         do {
121             if (actual.dato == valor) {
122                 if (actual == cabeza) {
123                     Node ultimo = cabeza;
124                     while (ultimo.siguiente != cabeza) {
125                         ultimo = ultimo.siguiente;
126                     }
127                     cabeza = cabeza.siguiente;
128                     ultimo.siguiente = cabeza;
129                 } else {
130                     anterior.siguiente = actual.siguiente;
131                 }
132                 return;
133             }
134             anterior = actual;
135             actual = actual.siguiente;
136         } while (actual != cabeza);
137
138         System.out.println("Valor no encontrado");
139     }
140
141     @Override
142     public static void main(String[] args) {
143         Scanner sc = new Scanner(System.in);
144         int opcion, valor, posicion;
145     }
146 }
```

Figura 4. Código Java del Ejercicio 2.



UNIVERSIDAD TÉCNICA DE AMBATO
FACULTAD DE INGENIERÍA EN SISTEMAS ELECTRÓNICA E INDUSTRIAL
CARRERA DE SOFTWARE
CICLO ACADÉMICO: JULIO 2026 – DICIEMBRE 2026



```
145 do {
146     System.out.println("\n-----");
147     System.out.println("  INSERCIÓN Y ELIMINACIÓN");
148     System.out.println("-----");
149     System.out.println("1. Insertar en posición específica");
150     System.out.println("2. Eliminar por posición");
151     System.out.println("3. Eliminar por valor");
152     System.out.println("4. Mostrar lista");
153     System.out.println("5. Salir");
154     System.out.print("Seleccione una opción: ");
155     opcion = sc.nextInt();
156
157     switch (opcion) {
158         case 1:
159             System.out.print("Valor a insertar: ");
160             valor = sc.nextInt();
161             System.out.print("Posición (0 = Inicio): ");
162             posicion = sc.nextInt();
163             System.out.println("Lista ANTES:");
164             mostrarLista();
165             insertarEnPosicion(valor, posicion);
166             System.out.println("Lista DESPUES:");
167             mostrarLista();
168             break;
169         case 2:
170             System.out.print("Posición a eliminar: ");
171             posicion = sc.nextInt();
172             System.out.println("Lista ANTES:");
173             mostrarLista();
174             eliminarPorPosicion(posicion);
175             System.out.println("Lista DESPUES:");
176             mostrarLista();
177             break;
178         case 3:
179             System.out.print("Valor a eliminar: ");
180             valor = sc.nextInt();
181             System.out.println("Lista ANTES:");
182             mostrarLista();
183             eliminarPorValor(valor);
184             System.out.println("Lista DESPUES:");
185             mostrarLista();
186             break;
187         case 4:
188             mostrarLista();
189             break;
190         case 5:
191             System.out.println("Saliendo...");
192             break;
193         default:
194             System.out.println("Opción inválida");
195     }
196 } while (opcion != 5);
197 sc.close();
198 }
199 }
```

Figura 5. Código Java del Ejercicio 2.



UNIVERSIDAD TÉCNICA DE AMBATO
FACULTAD DE INGENIERÍA EN SISTEMAS ELECTRÓNICA E INDUSTRIAL
CARRERA DE SOFTWARE
CICLO ACADÉMICO: JULIO 2026 – DICIEMBRE 2026



```
-----
INSERCIÓN Y ELIMINACIÓN
-----
1. Insertar en posición específica
2. Eliminar por posición
3. Eliminar por valor
4. Mostrar lista
5. Salir
Seleccione una opción: 1
Valor a insertar: 590
Posición (0 = inicio): 0
Lista ANTES:
Lista vacía
Lista DESPUES:
590 -> (vuelve a 590)
```

```
-----
INSERCIÓN Y ELIMINACIÓN
-----
1. Insertar en posición específica
2. Eliminar por posición
3. Eliminar por valor
4. Mostrar lista
5. Salir
Seleccione una opción: 1
Valor a insertar: 591
Posición (0 = inicio): 1
Lista ANTES:
590 -> (vuelve a 590)
Lista DESPUES:
590 -> 591 -> (vuelve a 590)
```

```
-----
INSERCIÓN Y ELIMINACIÓN
-----
1. Insertar en posición específica
2. Eliminar por posición
3. Eliminar por valor
4. Mostrar lista
5. Salir
Seleccione una opción: 3
Valor a eliminar: 590
Lista ANTES:
590 -> 591 -> (vuelve a 590)
Lista DESPUES:
591 -> (vuelve a 591)
```

```
-----
INSERCIÓN Y ELIMINACIÓN
-----
1. Insertar en posición específica
2. Eliminar por posición
3. Eliminar por valor
4. Mostrar lista
5. Salir
Seleccione una opción: 4
591 -> (vuelve a 591)
```

```
-----
INSERCIÓN Y ELIMINACIÓN
-----
1. Insertar en posición específica
2. Eliminar por posición
3. Eliminar por valor
4. Mostrar lista
5. Salir
Seleccione una opción: 5
Saliendo...
PS C:\Users\Jampier_M\Documents\Tarea_Listas\Tarea_Listas\scr_Java>
```

Figura 6. Ejecución del Ejercicio 2.



UNIVERSIDAD TÉCNICA DE AMBATO
FACULTAD DE INGENIERÍA EN SISTEMAS ELECTRÓNICA E INDUSTRIAL
CARRERA DE SOFTWARE
CICLO ACADÉMICO: JULIO 2026 – DICIEMBRE 2026



```
1 import java.util.Scanner;
2
3 class ProcesoModo {
4     String nombre;
5     int tiempoRestante;
6     ProcesoModo siguiente;
7
8     public ProcesoModo(String nombre, int tiempoRestante) {
9         this.nombre = nombre;
10        this.tiempoRestante = tiempoRestante;
11        this.siguiente = null;
12    }
13
14 }
15
16 public class Ejercicio_RoundRobin {
17     static ProcesoModo cabeza;
18     static final int QUANTUM = 2;
19
20     // Retorna true si no quedan procesos en la lista
21     public static boolean estaVacia() {
22         return cabeza == null;
23     }
24
25     // Agrega un nuevo proceso al final de la cola circular
26     public static void agregarProceso(String nombre, int tiempo) {
27         ProcesoModo nuevo = new ProcesoModo(nombre, tiempo);
28         if (estaVacia()) {
29             cabeza = nuevo;
30             nuevo.siguiente = cabeza;
31             return;
32         }
33         ProcesoModo actual = cabeza;
34         while (actual.siguiente != cabeza) {
35             actual = actual.siguiente;
36         }
37         actual.siguiente = nuevo;
38         nuevo.siguiente = cabeza;
39     }
40
41     // Imprime el estado actual de la cola de procesos
42     public static void mostrarEstado() {
43         if (estaVacia()) {
44             System.out.println("Lista vacía (no quedan procesos)");
45             return;
46         }
47         ProcesoModo actual = cabeza;
48         do {
49             System.out.print("[" + actual.nombre + " | " + actual.tiempoRestante + "] -> ");
50             actual = actual.siguiente;
51         } while (actual != cabeza);
52         System.out.println("vuelve a " + cabeza.nombre + " ]");
53     }
54
55     // Simulación Round Robin con quantum fijo de 2
56     public static void simular() {
57         if (estaVacia()) {
58             System.out.println("No hay procesos para simular");
59             return;
60         }
61
62         int turno = 1;
63         ProcesoModo actual = cabeza;
64         ProcesoModo anterior = null; // para poder eliminar el nodo actual
65
66         // se busca "anterior" en el último nodo antes de empezar
67         anterior = cabeza;
68         while (anterior.siguiente != cabeza) {
69             anterior = anterior.siguiente;
70         }
71
72         while (!estaVacia()) {
73             System.out.println("----- Turno " + turno + " -----");
74             System.out.println("Ejecutando: " + actual.nombre + " (tiempo restante antes: " + actual.tiempoRestante + ")");
75
76             int tiempoPorEjecutar = Math.min(QUANTUM, actual.tiempoRestante);
77             actual.tiempoRestante -= tiempoPorEjecutar;
78
79             System.out.println("Tiempo usado: " + tiempoPorEjecutar + " | Tiempo restante: " + actual.tiempoRestante);
80
81             if (actual.tiempoRestante <= 0) {
82                 System.out.println(actual.nombre + " terminó su ejecución y sale de la lista");
83
84                 if (actual == cabeza) {
85                     if (actual.siguiente == actual) {
86                         // era el único proceso
87                         cabeza = null;
88                         actual = null;
89                     } else {
90                         cabeza = actual.siguiente;
91                         anterior.siguiente = cabeza;
92                         actual = cabeza;
93                     }
94                 } else {
95                     anterior.siguiente = actual.siguiente;
96                     actual = anterior.siguiente;
97                 }
98             } else {
99                 System.out.println(actual.nombre + " no terminó, vuelve al final del ciclo");
100                anterior = actual;
101                actual = actual.siguiente;
102            }
103
104            System.out.println("Estado de la lista");
105            mostrarEstado();
106            turno++;
107        }
108
109        System.out.println("¡Todos los procesos han finalizado!");
110    }
111
112    Run[Main]
113    public static void main(String[] args) {
114        Scanner sc = new Scanner(System.in);
115        int opcion;
116
117        do {
118            System.out.println("-----");
119            System.out.println("1. ROUND ROBIN");
120            System.out.println("2. Agregar proceso");
121            System.out.println("3. Mostrar lista de procesos");
122            System.out.println("4. Iniciar simulación");
123            System.out.println("5. Salir");
124            System.out.print("Seleccione una opción: ");
125            opcion = sc.nextInt();
126
127            switch (opcion) {
128                case 1:
129                    System.out.print("Nombre del proceso: ");
130                    String nombre = sc.next();
131                    System.out.print("Tiempo restante: ");
132                    int tiempo = sc.nextInt();
133                    agregarProceso(nombre, tiempo);
134                    break;
135                case 2:
136                    mostrarEstado();
137                    break;
138                case 3:
139                    simular();
140                    break;
141                case 4:
142                    System.out.println("Saliendo...");
143                    break;
144                default:
145                    System.out.println("Opción inválida");
146            }
147        } while (opcion != 5);
148
149        sc.close();
150    }
151 }
```

Figura 7. Código Java del Ejercicio 3.



```
-----  
ROUND ROBIN  
-----  
1. Agregar proceso  
2. Mostrar lista de procesos  
3. Iniciar simulacion  
4. Salir  
Seleccione una opcion: 1  
Nombre del proceso: p1  
Tiempo restante: 8  
  
-----  
ROUND ROBIN  
-----  
1. Agregar proceso  
2. Mostrar lista de procesos  
3. Iniciar simulacion  
4. Salir  
Seleccione una opcion: 1  
Nombre del proceso: p2  
Tiempo restante: 4  
  
-----  
ROUND ROBIN  
-----  
1. Agregar proceso  
2. Mostrar lista de procesos  
3. Iniciar simulacion  
4. Salir  
Seleccione una opcion: 1  
Nombre del proceso: p3  
Tiempo restante: 3  
  
-----  
ROUND ROBIN  
-----  
1. Agregar proceso  
2. Mostrar lista de procesos  
3. Iniciar simulacion  
4. Salir  
Seleccione una opcion: 2  
[p1:8] -> [p2:4] -> [p3:3] -> (vuelve a p1)  
  
-----  
ROUND ROBIN  
-----  
1. Agregar proceso  
2. Mostrar lista de procesos  
3. Iniciar simulacion  
4. Salir  
Seleccione una opcion: 3  
  
--- Turno 1 ---  
Ejecutando: p1 (tiempo restante antes: 8)  
Tiempo usado: 2 | Tiempo restante: 6  
p1 no termino, vuelve al final del ciclo  
Estado de la lista:  
[p1:6] -> [p2:4] -> [p3:3] -> (vuelve a p1)
```

Figura 8. *Ejecución del Ejercicio 3.*



```
--- Turno 2 ---  
Ejecutando: p2 (tiempo restante antes: 4)  
Tiempo usado: 2 | Tiempo restante: 2  
p2 no termino, vuelve al final del ciclo  
Estado de la lista:  
[p1:6] -> [p2:2] -> [p3:3] -> (vuelve a p1)
```

```
--- Turno 3 ---  
Ejecutando: p3 (tiempo restante antes: 3)  
Tiempo usado: 2 | Tiempo restante: 1  
p3 no termino, vuelve al final del ciclo  
Estado de la lista:  
[p1:6] -> [p2:2] -> [p3:1] -> (vuelve a p1)
```

```
--- Turno 4 ---  
Ejecutando: p1 (tiempo restante antes: 6)  
Tiempo usado: 2 | Tiempo restante: 4  
p1 no termino, vuelve al final del ciclo  
Estado de la lista:  
[p1:4] -> [p2:2] -> [p3:1] -> (vuelve a p1)
```

```
--- Turno 5 ---  
Ejecutando: p2 (tiempo restante antes: 2)  
Tiempo usado: 2 | Tiempo restante: 0  
p2 termino su ejecucion y sale de la lista  
Estado de la lista:  
[p1:4] -> [p3:1] -> (vuelve a p1)
```

```
--- Turno 6 ---  
Ejecutando: p3 (tiempo restante antes: 1)  
Tiempo usado: 1 | Tiempo restante: 0  
p3 termino su ejecucion y sale de la lista  
Estado de la lista:  
[p1:4] -> (vuelve a p1)
```

```
--- Turno 7 ---  
Ejecutando: p1 (tiempo restante antes: 4)  
Tiempo usado: 2 | Tiempo restante: 2  
p1 no termino, vuelve al final del ciclo  
Estado de la lista:  
[p1:2] -> (vuelve a p1)
```

```
--- Turno 8 ---  
Ejecutando: p1 (tiempo restante antes: 2)  
Tiempo usado: 2 | Tiempo restante: 0  
p1 termino su ejecucion y sale de la lista  
Estado de la lista:  
Lista vacia (no quedan procesos)
```

Todos los procesos han finalizado.

```
-----  
ROUND ROBIN  
-----
```

1. Agregar proceso
2. Mostrar lista de procesos
3. Iniciar simulacion
4. Salir

Seleccione una opcion: 4

Saliendo...

PS C:\Users\Jampier_M\Documents\Tarea_Listas\Tarea_Listas\scr_Java> █

Figura 9. Ejecución del Ejercicio 3.



UNIVERSIDAD TÉCNICA DE AMBATO
FACULTAD DE INGENIERÍA EN SISTEMAS ELECTRÓNICA E INDUSTRIAL
CARRERA DE SOFTWARE
CICLO ACADÉMICO: JULIO 2026 – DICIEMBRE 2026



```
1  import java.util.Scanner;
2
3  class PersonaNodo {
4      int id;
5      PersonaNodo siguiente;
6
7      public PersonaNodo(int id) {
8          this.id = id;
9          this.siguiente = null;
10     }
11 }
12
13 public class Ejercicio4_Josephus {
14
15     // Arma el círculo de n personas y elimina cada k-esima hasta dejar 1 sola
16     public static void resolverJosephus(int n, int k) {
17         PersonaNodo cabeza = new PersonaNodo(1);
18         PersonaNodo actual = cabeza;
19
20         // se construye el círculo con n personas
21         for (int i = 2; i <= n; i++) {
22             actual.siguiente = new PersonaNodo(i);
23             actual = actual.siguiente;
24         }
25         actual.siguiente = cabeza; // se cierra el círculo
26
27         System.out.println("\n--- Josephus con n=" + n + ", k=" + k + " ---");
28         System.out.print(s: "Orden de eliminacion: ");
29
30         PersonaNodo anterior = actual; // apunta al ultimo (anterior a cabeza)
31         actual = cabeza;
32
33         while (actual.siguiente != actual) {
34             // avanzar k-1 posiciones para llegar a la k-esima persona
35             for (int i = 1; i < k; i++) {
36                 anterior = actual;
37                 actual = actual.siguiente;
38             }
39             System.out.print(actual.id + " ");
40             anterior.siguiente = actual.siguiente; // se elimina "actual"
41             actual = anterior.siguiente;
42         }
43
44         System.out.println("\nSuperviviente final: " + actual.id);
45     }
46
47     Run | Debug
48     public static void main(String[] args) {
49         Scanner sc = new Scanner(System.in);
50         int opcion;
51
52         do {
53             System.out.println(x: "\n-----");
54             System.out.println(x: "      PROBLEMA DE JOSEPHUS");
55             System.out.println(x: "-----");
56             System.out.println(x: "1. Ejecutar caso n=5, k=2");
57             System.out.println(x: "2. Ejecutar caso n=7, k=3");
58             System.out.println(x: "3. Ingresar n y k personalizados");
59             System.out.println(x: "4. Salir");
60             System.out.print(s: "Seleccione una opcion: ");
61             opcion = sc.nextInt();
62
63             switch (opcion) {
64                 case 1:
65                     resolverJosephus(n: 5, k: 2);
66                     break;
67                 case 2:
68                     resolverJosephus(n: 7, k: 3);
69                     break;
70                 case 3:
71                     System.out.print(s: "Ingrese n: ");
72                     int n = sc.nextInt();
73                     System.out.print(s: "Ingrese k: ");
74                     int k = sc.nextInt();
75                     resolverJosephus(n, k);
76                     break;
77                 case 4:
78                     System.out.println(x: "Saliendo...");
79                     break;
80                 default:
81                     System.out.println(x: "Opcion invalida");
82             }
83         } while (opcion != 4);
84
85         sc.close();
86     }
87 }
```

Figura 10. Código Java del Ejercicio 4.



```
-----  
PROBLEMA DE JOSEPHUS  
-----  
1. Ejecutar caso n=5, k=2  
2. Ejecutar caso n=7, k=3  
3. Ingresar n y k personalizados  
4. Salir  
Seleccione una opcion: 1
```

```
--- Josephus con n=5, k=2 ---  
Orden de eliminacion: 2 4 1 5  
Superviviente final: 3
```

```
-----  
PROBLEMA DE JOSEPHUS  
-----  
1. Ejecutar caso n=5, k=2  
2. Ejecutar caso n=7, k=3  
3. Ingresar n y k personalizados  
4. Salir  
Seleccione una opcion: 2
```

```
--- Josephus con n=7, k=3 ---  
Orden de eliminacion: 3 6 2 7 5 1  
Superviviente final: 4
```

```
-----  
PROBLEMA DE JOSEPHUS  
-----  
1. Ejecutar caso n=5, k=2  
2. Ejecutar caso n=7, k=3  
3. Ingresar n y k personalizados  
4. Salir  
Seleccione una opcion: 3  
Ingrese n: 1  
Ingrese k: 8
```

```
--- Josephus con n=1, k=8 ---  
Orden de eliminacion:  
Superviviente final: 1
```

```
-----  
PROBLEMA DE JOSEPHUS  
-----  
1. Ejecutar caso n=5, k=2  
2. Ejecutar caso n=7, k=3  
3. Ingresar n y k personalizados  
4. Salir  
Seleccione una opcion: 4  
Saliendo...  
PS C:\Users\Jampier_M\Documents\Tarea_Listas\Tarea_Listas\scr_Java> |
```

Figura 11. Ejecución del Ejercicio 4.



UNIVERSIDAD TÉCNICA DE AMBATO
FACULTAD DE INGENIERÍA EN SISTEMAS ELECTRÓNICA E INDUSTRIAL
CARRERA DE SOFTWARE
CICLO ACADÉMICO: JULIO 2026 – DICIEMBRE 2026



```
1 import java.util.Scanner;
2
3 class CancionModo {
4     String nombre;
5     CancionModo siguiente;
6
7     public CancionModo(String nombre) {
8         this.nombre = nombre;
9         this.siguiente = null;
10    }
11 }
12
13 public class Ejercicio5_PlaylistCircular {
14
15     static CancionModo cabeza;
16     static CancionModo actual; // canción que se esta "reproduciendo"
17
18     // Retorna true si la playlist no tiene canciones
19     public static boolean estaVacia() {
20         return cabeza == null;
21     }
22
23     // Agrega la canción al inicio y la convierte en la nueva cabeza
24     public static void agregarInicio(String nombre) {
25         CancionModo nueva = new CancionModo(nombre);
26         if (estaVacia()) {
27             cabeza = nueva;
28             nueva.siguiente = cabeza;
29             actual = cabeza;
30             return;
31         }
32         CancionModo ultimo = cabeza;
33         while (ultimo.siguiente != cabeza) {
34             ultimo = ultimo.siguiente;
35         }
36         nueva.siguiente = cabeza;
37         ultimo.siguiente = nueva;
38         cabeza = nueva;
39     }
40
41     // Agrega la canción al final, sin cambiar la cabeza
42     public static void agregarFinal(String nombre) {
43         CancionModo nueva = new CancionModo(nombre);
44         if (estaVacia()) {
45             cabeza = nueva;
46             nueva.siguiente = cabeza;
47             actual = cabeza;
48             return;
49         }
50         CancionModo ultimo = cabeza;
51         while (ultimo.siguiente != cabeza) {
52             ultimo = ultimo.siguiente;
53         }
54         ultimo.siguiente = nueva;
55         nueva.siguiente = cabeza;
56     }
57
58     // Recorre la playlist marcando cual canción se esta reproduciendo
59     public static void mostrarPlaylist() {
60         if (estaVacia()) {
61             System.out.println("Playlist vacia");
62             return;
63         }
64         CancionModo nodo = cabeza;
65         do {
66             String marca = (nodo == actual) ? " (reproduciendo)" : "";
67             System.out.print(nodo.nombre + marca + " -> ");
68             nodo = nodo.siguiente;
69         } while (nodo != cabeza);
70         System.out.println("(vuelve a " + cabeza.nombre + ")");
71     }
72
73     // Al llegar al final, "siguiente" apunta de nuevo a cabeza automaticamente
74     public static void reproducirSiguiente() {
75         if (estaVacia()) {
76             System.out.println("Playlist vacia, no hay nada que reproducir");
77             return;
78         }
79         actual = actual.siguiente;
80         System.out.println("Reproduciendo ahora: " + actual.nombre);
81     }
82
83     // Busca la canción por nombre y la desconecta de la lista
84     public static void eliminarPorNombre(String nombre) {
85         if (estaVacia()) {
86             System.out.println("Playlist vacia");
87             return;
88         }
89         if (cabeza.siguiente == cabeza) {
90             if (cabeza.nombre.equals(nombre)) {
91                 cabeza = null;
92                 actual = null;
93             } else {
94                 System.out.println("Cancion no encontrada");
95             }
96             return;
97         }
98         CancionModo nodo = cabeza;
99         CancionModo anterior = null;
100
101         do {
102             if (nodo.nombre.equals(nombre)) {
103                 if (nodo == actual) {
104                     actual = nodo.siguiente; // si se elimina la actual, pasa a la siguiente
105                 }
106                 if (nodo == cabeza) {
107                     CancionModo ultimo = cabeza;
108                     while (ultimo.siguiente != cabeza) {
109                         ultimo = ultimo.siguiente;
110                     }
111                     cabeza = cabeza.siguiente;
112                     ultimo.siguiente = cabeza;
113                 } else {
114                     anterior.siguiente = nodo.siguiente;
115                 }
116                 System.out.println(nombre + " eliminada de la playlist");
117                 return;
118             }
119             anterior = nodo;
120             nodo = nodo.siguiente;
121         } while (nodo != cabeza);
122     }
123 }
```

Figura 12. Código Java del Ejercicio 5.



UNIVERSIDAD TÉCNICA DE AMBATO
FACULTAD DE INGENIERÍA EN SISTEMAS ELECTRÓNICA E INDUSTRIAL
CARRERA DE SOFTWARE
CICLO ACADÉMICO: JULIO 2026 – DICIEMBRE 2026



```
121         anterior = nodo;
122         nodo = nodo.siguiente;
123     } while (nodo != cabeza);
124
125     System.out.println(x: "Cancion no encontrada");
126 }
127
128 Run[Debug
129 public static void main(String[] args) {
130     Scanner sc = new Scanner(System.in);
131     int opcion;
132
133     do {
134         System.out.println(x: "\n-----");
135         System.out.println(x: "      PLAYLIST CIRCULAR");
136         System.out.println(x: "-----");
137         System.out.println(x: "1. Agregar cancion al inicio");
138         System.out.println(x: "2. Agregar cancion al final");
139         System.out.println(x: "3. Mostrar playlist completo");
140         System.out.println(x: "4. Reproducir siguiente cancion");
141         System.out.println(x: "5. Eliminar cancion por nombre");
142         System.out.println(x: "6. Salir");
143         System.out.print(x: "Seleccione una opcion: ");
144         opcion = sc.nextInt();
145
146         switch (opcion) {
147             case 1:
148                 System.out.print(x: "Nombre de la cancion: ");
149                 agregarInicio(sc.next());
150                 break;
151             case 2:
152                 System.out.print(x: "Nombre de la cancion: ");
153                 agregarFinal(sc.next());
154                 break;
155             case 3:
156                 mostrarPlaylist();
157                 break;
158             case 4:
159                 reproducirSiguiente();
160                 break;
161             case 5:
162                 System.out.print(x: "Nombre de la cancion a eliminar: ");
163                 eliminarPorNombre(sc.next());
164                 break;
165             case 6:
166                 System.out.println(x: "Saliendo...");
167                 break;
168             default:
169                 System.out.println(x: "Opcion invalida");
170         }
171     } while (opcion != 6);
172
173     sc.close();
174 }
```

Figura 13. Código Java del Ejercicio 5.



UNIVERSIDAD TÉCNICA DE AMBATO
FACULTAD DE INGENIERÍA EN SISTEMAS ELECTRÓNICA E INDUSTRIAL
CARRERA DE SOFTWARE
CICLO ACADÉMICO: JULIO 2026 – DICIEMBRE 2026



```
-----
PLAYLIST CIRCULAR
-----
1. Agregar cancion al inicio
2. Agregar cancion al final
3. Mostrar playlist completa
4. Reproducir siguiente cancion
5. Eliminar cancion por nombre
6. Salir
Seleccione una opcion: 1
Nombre de la cancion: PUMAS

-----
PLAYLIST CIRCULAR
-----
1. Agregar cancion al inicio
2. Agregar cancion al final
3. Mostrar playlist completa
4. Reproducir siguiente cancion
5. Eliminar cancion por nombre
6. Salir
Seleccione una opcion: 1
Nombre de la cancion: RAIN

-----
PLAYLIST CIRCULAR
-----
1. Agregar cancion al inicio
2. Agregar cancion al final
3. Mostrar playlist completa
4. Reproducir siguiente cancion
5. Eliminar cancion por nombre
6. Salir
Seleccione una opcion: 2
Nombre de la cancion: GPS

-----
PLAYLIST CIRCULAR
-----
1. Agregar cancion al inicio
2. Agregar cancion al final
3. Mostrar playlist completa
4. Reproducir siguiente cancion
5. Eliminar cancion por nombre
6. Salir
Seleccione una opcion: 3
RAIN -> PUMAS (reproduciendo) -> GPS -> (vuelve a RAIN)

-----
PLAYLIST CIRCULAR
-----
1. Agregar cancion al inicio
2. Agregar cancion al final
3. Mostrar playlist completa
4. Reproducir siguiente cancion
5. Eliminar cancion por nombre
6. Salir
Seleccione una opcion: 4
Reproduciendo ahora: GPS
```

Figura 14. *Ejecución del Ejercicio 5.*



```
-----  
PLAYLIST CIRCULAR  
-----  
1. Agregar cancion al inicio  
2. Agregar cancion al final  
3. Mostrar playlist completa  
4. Reproducir siguiente cancion  
5. Eliminar cancion por nombre  
6. Salir  
Seleccione una opcion: 5  
Nombre de la cancion a eliminar: PUMAS  
PUMAS eliminada de la playlist  
  
-----  
PLAYLIST CIRCULAR  
-----  
1. Agregar cancion al inicio  
2. Agregar cancion al final  
3. Mostrar playlist completa  
4. Reproducir siguiente cancion  
5. Eliminar cancion por nombre  
6. Salir  
Seleccione una opcion: 3  
RAIN -> GPS (reproduciendo) -> (vuelve a RAIN)  
  
-----  
PLAYLIST CIRCULAR  
-----  
1. Agregar cancion al inicio  
2. Agregar cancion al final  
3. Mostrar playlist completa  
4. Reproducir siguiente cancion  
5. Eliminar cancion por nombre  
6. Salir  
Seleccione una opcion: 6  
Saliendo...  
PS C:\Users\Jampier_M\Documents\Tarea_Listas\Tarea_Listas\scr_Java>
```

Figura 15. *Ejecución del Ejercicio 5.*

En el siguiente enlace al repositorio, se encuentran disponibles para su visualización los códigos desarrollados Java. Asimismo, se adjuntan las evidencias del diagrama UML correspondiente y el presente informe en formatos .pdf.

Enlace repositorio: https://github.com/jampiercamacho94-cloud/Tarea_Listas.git